REACHABILITY-CONSTRAINED MODULAR CONTINUUM ROBOT OPTIMIZATION VIA SURROGATE MODELS

A PREPRINT

Mubai Gao
Department of Mechanical and Industrial Engineering
University of Toronto
Toronto, ON
mubai.gao@mail.utoronto.ca

Hussein Hassanali
Department of Mechanical and Industrial Engineering
University of Toronto
Toronto, ON
hussein.hassanali@mail.utoronto.ca

Rozilyn Marco
Department of Mechanical and Industrial Engineering
University of Toronto
Toronto, ON
rozilyn.marco@mail.utoronto.ca

Adam Stechishin
Department of Mechanical and Industrial Engineering
University of Toronto
Toronto, ON
adam.stechishin@mail.utoronto.ca

December 20, 2025

ABSTRACT

This paper investigates discrete optimization of modular tendon-driven continuum robot designs subject to reachability constraints. The design space is defined by discrete unit selections and segment boundaries, extending prior continuous optimization formulations to modular architectures. Reachability is estimated using piecewise-constant curvature kinematic simulation combined with inverse-kinematics refinement in cluttered synthetic workspaces. To reduce the computational burden of repeated reachability evaluation, surrogate models are trained to approximate reachability and multiple objective functions and are subsequently embedded within optimization and evolutionary search methods. Simulation results demonstrate that the proposed framework enables efficient exploration of large discrete design spaces for continuum robot design.

Keywords Continuum Robotics · Tendon-Driven Robots · Modular Robot Design · Discrete Optimization · Reachability Analysis · Surrogate-Assisted Optimization

1 Introduction

Continuum robots (CRs) offer unique advantages over traditional rigid-link manipulators, including intrinsic compliance, high dexterity, and the ability to operate effectively in constrained environments. These characteristics have motivated applications in minimally invasive surgery, industrial inspection, and confined-space manipulation (Russo et al. [2023a]). Designing CRs is difficult because their geometry, actuation, and deformation are strongly coupled in complex, nonlinear ways. Tendon-driven continuum robots (TDCRs) in particular enable full 3D shape control while maintaining minimal diameters, but their design spaces are high-dimensional and nonconvex.

TDCR design parameters often include segment lengths, tendon placement, and backbone stiffness. Traditionally, TDCR designs have been formulated over continuous design spaces for these parameters, subject to material and fabrication constraints. Recent work has introduced modular TDCR architectures, in which manipulators are assembled from a set of predefined structural units with fixed physical properties (Dewi et al. [2024]). Assembling a TDCR from predefined units discretizes the underlying design space, converting continuous geometric choices into a combinatorial selection of module types and sequences. Introduces modularity enables reduced fabrication effort through reuse of standardized components, rapid task-specific customization via reconfiguration of unit sequences and segmentation, and unit-level repair by replacing individual modules rather than the entire manipulator. Existing TDCR optimization

methods focus primarily on continuous geometric design parameters under reachability constraints (e.g., Cheong et al. [2021], Russo et al. [2023b] and Jabari et al. [2025]). In contrast, the discrete, combinatorial design space induced by modular TDCR architectures has not been systematically studied, the gap this project aims to address through design optimization.

The goal of this work is to identify modular TDCR configurations that balance workspace reachability constraints with physical performance criteria such as structural stiffness, compactness, actuation effort, and mass. We optimize over (i) the number of units used, (ii) the choice of unit type at each position, (iii) the number of segments, and (iv) the placement of segment boundaries. Consistent with prior work by Cheong et al. [2021], candidate designs must satisfy a minimum task-space reachability threshold of 0.95 to be considered feasible. Reachability is evaluated using Monte Carlo methods commonly employed in the literature, which require large numbers of repeated kinematic simulations. As prior work Bodily et al. [2017] recommends two to four million samples, direct evaluation of design reachability over the task space is computationally expensive. To address this challenge and improve optimization runtimes, we incorporate surrogate modeling to approximate reachability metrics of candidate designs, enabling efficient integration with solver-based and evolutionary optimization methods. By combining kinematic simulation, surrogate-assisted optimization, and discrete search, we demonstrate an end-to-end framework for modular continuum robot design with reachability constraints. We use both evolutionary optimization as done in prior TDCR design optimizations as well as investigate the viability of solver-based optimization for modular continuum robot design.

2 Related Work

Design optimization of tendon-driven continuum robots (TDCRs) has been studied with objectives of improving workspace coverage, actuation efficiency, and structural performance. Cheong et al. [2021] introduced a systematic framework for reachability-constrained TDCR optimization, demonstrating that evolutionary algorithms can significantly reduce manual design time, from on the order of weeks to a few hours. This work established evolutionary optimization as a practical tool for handling the highly nonconvex and simulation-driven nature of TDCR design. Subsequent studies extended this approach by incorporating additional physical and task-related constraints. Russo et al. [2023b] optimized TDCR geometry under tendon routing and stiffness constraints to improve workspace coverage and mechanical performance, while Jabari et al. [2025] focused on static workspace feasibility under load-bearing requirements. Despite differences in modeling assumptions and objectives, these approaches share several common characteristics: they operate over continuous geometric design parameters, assume a fixed number of segments, and rely predominantly on evolutionary optimization methods such as genetic algorithms or estimation-of-distribution algorithms.

The reliance on evolutionary algorithms in prior TDCR design work is largely driven by the computational structure of reachability evaluation. Monte Carlo–based reachability analysis typically requires thousands to millions of forward kinematic simulations Bodily et al. [2017], resulting in noisy, nonconvex objective landscapes that are not well suited to gradient-based optimization. Additionally, this evaluation procedure does not integrate with exact discrete or mixed-integer optimization methods such as Gurobi. Surrogate modeling techniques have been shown to accelerate expensive optimization loops by learning approximations of simulation-based objective functions; for instance, Forrester and Keane [2008] survey the use of Gaussian process response surfaces for engineering design optimization. More recently, neural surrogate models have been leveraged to predict objective function values for combinatorial optimization problems, reducing evaluation cost Xie et al. [2019].

These learning-enhanced optimization approaches suggest that incorporating surrogate modeling can improve the computational tractability of reachability-constrained TDCR design optimization by reducing the cost of reachability evaluation. By replacing expensive Monte Carlo–based simulation with surrogate-assisted reachability prediction, such models enable scalable exploration of modular and discrete TDCR design spaces that would otherwise be computationally prohibitive. In this way, the present work extends prior TDCR design optimization methodologies beyond traditional continuous, evolutionary-only frameworks toward hybrid ML-driven optimization approaches capable of interfacing with discrete and mixed-integer formulations for modular continuum robot architectures.

3 Preliminaries

Kinematic Modeling of Tendon-Driven Continuum Robots

Most continuum robot design frameworks rely on the piecewise-constant curvature (PCC) assumption, in which each segment bends with constant curvature and a fixed bending-plane orientation. Under this assumption, the configuration of a segment is parameterized by a curvature magnitude κ and a bending-plane angle ϕ. The PCC formulation provides closed-form forward kinematics (FK) expressions mapping (κ, ϕ) to a homogeneous transformation describing the segment tip pose Webster and Jones [2010].

For a segment of length L, the closed-form transformation $T(\kappa, \phi, L) \in SE(3)$ is given by

$$T = \begin{bmatrix} R(\kappa, \phi, L) & p(\kappa, \phi, L) \\ \mathbf{0}^\top & 1 \end{bmatrix},$$

where $p \in \mathbb{R}^3$ and $R \in SO(3)$ denote the tip position and orientation, respectively. In the limiting case $\kappa \to 0$, the segment reduces to a straight backbone of length L.

Heterogeneous Segment Modeling

As the design space considered in this work includes heterogeneous modular units, the PCC formulation is applied on a per-segment basis. The curvature generated within a segment depends on the bending stiffness of the constituent units. From Euler–Bernoulli beam theory, the curvature κ induced by a bending moment M is

$$\kappa = \frac{M}{EI},$$

where EI denotes the bending stiffness. For a fixed tendon displacement, the applied bending moment is approximately constant across all units in a segment. Consequently, curvature scales inversely with stiffness (derivation in Appendix A).

Let κ_s denote the curvature parameter sampled for segment s, defined with respect to the least stiff unit in that segment. For any unit t within the same segment, curvature is scaled according to

$$\kappa_t = \kappa_s \frac{EI_s}{EI_t}.$$

This scaling rule ensures physically consistent deformation across heterogeneous segments.

Forward Kinematics Across Modular Units

The full robot forward kinematics is obtained by chaining unit-level transformations along the backbone. For a design consisting of N active units, the end-effector transformation is computed as

$$T_{\text{ee}} = \prod_{i=1}^{N} T_i(\kappa_i, \phi_{s(i)}, L_i),$$

where $s(i)$ denotes the segment index associated with unit i. Segment boundaries determine where curvature and bending-plane parameters change along the chain.

Inverse Kinematics Refinement

To refine configurations near the boundary of feasibility, inverse kinematics (IK) is performed using a damped least-squares (DLS) approach. Given a task-space position error $\mathbf{e} \in \mathbb{R}^3$ and the Jacobian $J \in \mathbb{R}^{3\times 2S}$ relating curvature and bending-plane parameters to tip position, the DLS update is computed as

$$\Delta\mathbf{q} = J^\top (JJ^\top + \lambda^2 I)^{-1}\mathbf{e},$$

where λ is a damping coefficient. The Jacobian is computed numerically via finite differences of the FK map. This IK procedure is used to locally refine sampled configurations and improve reachability estimates, following the approach of Cheong et al. [2021].

4 Methodology

4.1 Modular Design Encoding

A modular TDCR design is represented by a discrete design vector

$$D = [\mathbf{u}, \mathbf{b}],$$

where $\mathbf{u} = (u_1, \ldots, u_{N_{\max}})$ encodes the unit type placed at each position along the backbone, and $\mathbf{b} = (b_1, \ldots, b_{N_{\max}})$ encodes the placement of segment boundaries. Figure 1 illustrates the discrete encoding used to represent modular TDCR designs.

Each design variable $u_i \in \{0, 1, 2, 3\}$ specifies the unit type at position i. A value of $u_i = 0$ indicates an empty slot, while values 1 through 3 correspond to predefined module types with fixed geometric and mechanical properties as

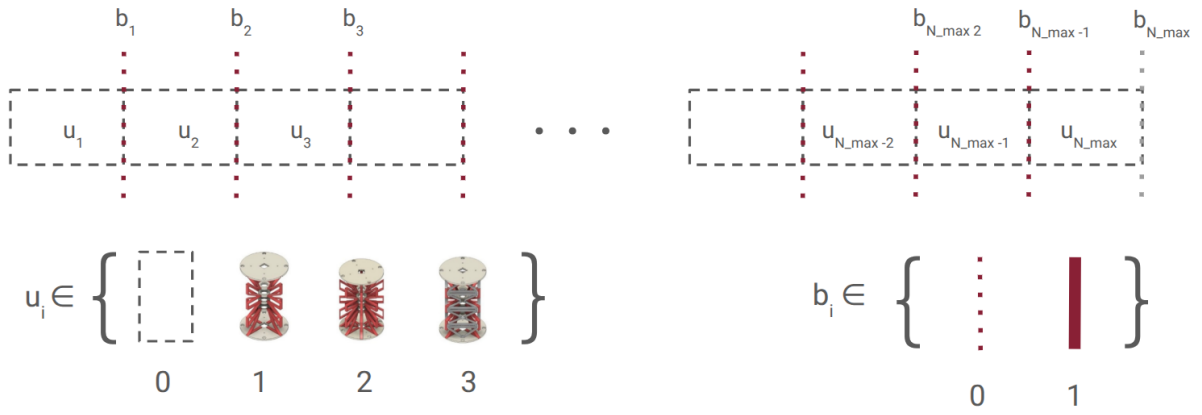


Figure 1: Discrete modular design encoding for the TDCR. u_1 is the beginning or proximal unit of the design and $u_{N_{max}}$ is the final or distal unit of the design. Unit-type variable $u_i \in \{0, 1, 2, 3\}$, where $u_i = 0$ denotes an empty slot and values 1–3 correspond to predefined module types with fixed geometric and mechanical properties.

defined in Dewi et al. [2024] and listed in Appendix B. Segment boundaries are encoded by binary variables $b_i \in \{0, 1\}$, where $b_i = 1$ denotes a boundary between positions i and $i + 1$. Boundaries are only permitted between active units, ensuring that each segment consists of a contiguous sequence of modules.

Actuation variables are not part of the design vector. For a fixed design D, actuation is parameterized at the segment level using the PCC formulation. Each segment s is controlled by a curvature magnitude κ_s and a bending plane angle ϕ_s. These actuation parameters are sampled during reachability evaluation and marginalized when computing design-level objective values.

4.2 Reachability Evaluation

Design feasibility is determined by a reachability metric $\Theta(D) \in [0, 1]$, defined as the fraction of task-space voxels that can be reached by the robot tip within a specified tolerance. Following Cheong et al. Cheong et al. [2021], reachability is evaluated by sampling segment-level actuation parameters (κ_s, ϕ_s), or robot configurations within its configuration space. Using these samples forward kinematics using a PCC model, and refining near-boundary configurations using inverse kinematics. A design is considered feasible if its reachability satisfies

$$\Theta(D) \geq \alpha,$$

where the minimum reachability threshold is set to $\alpha = 0.95$. The resulting reachable voxel set and corresponding feasible configurations are used both for ground-truth evaluation and as training data for surrogate models.

4.3 Objective Functions

Each modular design is evaluated using four objective functions: structural sag $f_1(D)$ based on an Euler–Bernoulli beam approximation, compactness $f_2(D)$ defined as total backbone length, actuation effort $f_3(D)$ measured by average curvature magnitude over feasible configurations, and total mass $f_4(D)$ defined as the sum of unit weights. These objectives capture competing tradeoffs between stiffness, reachability, actuation efficiency, and material usage. Optimization is performed either on individual objectives or on a weighted composite objective

$$f(D) = w_1 f_1(D) + w_2 f_2(D) + w_3 f_3(D) + w_4 f_4(D),$$

where the weights are selected to balance physical performance criteria. The full mathematical expressions for each objective are provided in Appendix C.

4.4 Optimization Framework

Two complementary optimization approaches are investigated to explore the discrete modular design space. The first approach employs evolutionary optimization methods, which are well suited to highly nonconvex and simulation-driven problems. The second approach formulates the design problem as a discrete optimization program using a commercial solver, enabled by surrogate models that approximate reachability and actuation effort. We adopt a predict–then–optimize formulation, in which surrogate models are trained offline to approximate reachability and objective terms and are subsequently embedded as fixed predictors within optimization algorithms.

4.5 Surrogate-Assisted Optimization

To enable efficient optimization, surrogate models are trained to predict both the composite objective $f(D)$ and the reachability metric $\Theta(D)$ from the discrete design encoding. Each design is represented as a fixed-length feature vector derived from the unit-type and segmentation variables.

Multiple surrogate model architectures are evaluated, including Gaussian process regression, fully connected neural networks, residual multilayer perceptrons, one-dimensional convolutional networks, and graph neural networks. These models span unstructured baselines and structure-aware architectures that explicitly exploit the sequential nature of modular TDCR designs.

Solver-Based Optimization Implementation

The discrete design optimization problem is formulated as a binary integer nonlinear program, as detailed in Appendix D. Discrete design variables corresponding to unit-type selection and segment boundary placement were modeled as binary decision variables, and structural constraints were added directly to the solver using linear expressions.

Nonlinear and simulation-derived quantities, including reachability and actuation effort, were incorporated using surrogate predictors to maintain compatibility with mixed-integer optimization. These surrogate models were embedded into the optimization problem using the Gurobi Machine Learning library, enabling a predict-then-optimize formulation. Although this approach allows candidate designs to be evaluated efficiently, complex surrogate models such as convolutional neural networks could not be used due to compatibility issues with Gurobi Machine Learning. For this reason, simple models such as linear regression, decision trees, and feed-forward neural networks with ReLU activation will be used in the integer programming formulation. To ensure that solver-based approaches were comparable with EDA/GA, a feed forward neural network was used in these approaches.

4.6 Evolutionary Optimization

Evolutionary optimization is performed using both a Genetic Algorithm (GA) and an Estimation of Distribution Algorithm (EDA). In both methods, candidate designs are encoded using the discrete vectors $\mathbf{u}$ and $\mathbf{b}$. Fitness is evaluated using the weighted objective $f(D)$, augmented by a quadratic penalty for reachability violations:

$$F(D) = f(D) + \lambda \max(0, \alpha - \Theta(D))^2.$$

The GA evolves the population through selection, crossover, and mutation, enabling structural recombination and stochastic exploration of the design space. The EDA constructs a probabilistic model of high-performing candidates and samples new designs from the learned distribution. To mitigate premature convergence in the EDA, diversity-enhancing mechanisms such as random resampling and temperature decay are incorporated.

5 Experimental Setup

Dataset Generation. All datasets used in this study were synthetically generated using an in-house kinematic simulator for modular tendon-driven continuum robots. Robot designs were sampled uniformly from the discrete design space defined by the encoding $D = [\mathbf{u}, \mathbf{b}]$. For each design, reachability $\Theta(D)$ was evaluated over a discretized workspace, and the corresponding objective values f_1–f_4 were computed.

Reachability was estimated by sampling 500,000 forward kinematics configurations per design, which provided a balance between computational cost and estimation accuracy. The resulting dataset consists of 1,810 design instances, stored as tuples $(D, \Theta, f_1, f_2, f_3, f_4)$. This dataset was used for surrogate model training as well as for evaluating optimization-based design methods.

Baselines. Two baselines were considered for comparison. The first baseline is the best feasible design obtained from random sampling during dataset generation i.e the reachability constraint is satisfied and for each objective (composite or individual) the design has the lowest objective value. The second baseline is a manually specified reference design consisting of three homogeneous segments with monotonically increasing stiffness from the distal to proximal end i.e. $\mathbf{u}$ =[3 3 3 2 2 2 1 1 1 1] and $\mathbf{b}$ =[0 0 1 0 0 1 0 0 0 0].

Evaluation Metrics. All methods were evaluated using the individual objective function values as well as the value of the weighted $f(D)$ with $w_1 = 0.5, w_2 = 0.1, w_3 = 0.3, w_4 = 0.1$. The optimality gap is computed as $(f_{\text{method}} - f_{\text{best}})/f_{\text{best}}$, where f_{best} is the lowest feasible objective value across all methods. Runtime was additionally recorded to assess computational efficiency across optimization approaches.

Surrogate Model Training. Surrogate models were trained using the generated dataset to predict reachability and objective values from the design encoding. Inputs were standardized to zero mean and unit variance, and data were split

into 80% training and 20% validation sets. Gaussian process models were implemented using scikit-learn, while neural network models (ANN, ResMLP, CNN, and GNN) were implemented in PyTorch, with the GNN implemented using PyTorch Geometric. All models were trained using the Adam optimizer and mean squared error loss.

Evolutionary Optimization Setup. The genetic algorithm (GA) was implemented with a population size of 40, a crossover rate of 0.7, a mutation rate of 0.2, and a maximum of 800 fitness evaluations. The estimation of distribution algorithm (EDA) used a population size of 50 and a maximum of 1,000 evaluations. Both methods were evaluated in the same discretized workspace environment with identical target voxel distributions to ensure fair comparison. Each algorithm was run 20 times to obtain the mean and standard deviation of the results. Both the EDA and GA methods successfully generated feasible manipulator configurations by embedding the neural network (NN) surrogate model for reachability and f_3 estimation into the optimization loop. The imposed penality for desigs that do not meet the reachability threshold is 1e7.

Gurobi-Based Optimization Experiments. Separate optimization models were constructed using different surrogate classes, including linear regression, regression trees, random forests, and neural networks. Due to prohibitively high runtimes observed when using random forests and neural networks for both quantities, a hybrid configuration was adopted in which a neural network was used to model actuation effort and a regression tree was used to model reachability. All surrogate models were trained using the randomly generated design dataset, with an 80%/20% train–test split.

Computing Environment. All work was done on the Google Colab Python 3 TPU runtime.

6 Results

6.1 Surrogate Model

The CNN achieves near-perfect prediction of $f(D)$ and strong reachability accuracy. ResMLP significantly improves over a standard ANN, while the GNN performs similarly well on θ, suggesting that reachability behavior is relational across segments. The GP baseline performs worst due to lack of hierarchical feature learning. Table 1 summarizes the surrogate performance when predicting the composite objective $f(D)$ and reachability $\theta(D)$.

Table 1: Surrogate model performance on real simulator data (validation set).

Model	R_f^2	MAE_f	R_θ^2	MAE_θ
Gaussian Process	0.470	2.663	0.604	0.124
Regular ANN	0.7595	1.7285	0.9013	0.0671
ResMLP	0.899	1.166	0.967	0.032
CNN	**0.999**	**0.118**	**0.986**	**0.020**
GNN	0.889	1.328	0.981	0.027

To test surrogate sensitivity to objective decomposition, we trained ANN and CNN models to predict only the f_3 component rather than the composite objective. Performance decreased substantially for both networks.

Table 2: Surrogate performance when predicting only f_3.

Model	$R_{f_3}^2$	MAE_{f_3}
ANN (f_3 only)	0.2199	0.0879
CNN (f_3 only)	0.5242	0.0612

While CNN still outperforms ANN, both show significantly weaker results compared to composite-objective training. This suggests that f_3 contains sharper or less redundant structure that is harder to approximate directly. In contrast, the combined objective $f(D)$ smooths conflicting terms and creates learning redundancy, making it more predictable.

For integration into the optimization algorithms, several simple machine learning models were trained. The coefficient of determination was computed for each model, and is displayed in Table 3. Based on these results, the random forests and neural networks were used.

Running the optimization model in Gurobi with different surrogate models and using the weighted objective function $f(D)$ yielded the results seen in Table 4. The models that used linear regression and decision trees as proxies for reachability and actuation effort were solved quickly, taking under 2 seconds to run. High running times were observed when using random forests and neural networks, yet combining neural networks and decision trees resulted in models

Table 3: Coefficient of determination of models used in the BINLP.

Model	Training R^2 (θ)	Testing R^2 (θ)	Training R^2 (f_3)	Testing R^2 (f_3)
Linear Regression	0.6956	0.6534	0.4580	0.3744
Regression Tree	1.0000	0.9792	0.9996	0.2286
Random Forest Regressor	0.9985	0.9904	0.9464	0.6153
Neural Network	0.999	0.984	0.988	0.544

that were solvable in approximately 12 minutes with an optimality gap of 1.5%. This was unexpected, since using the same objective functions with different weights yielded good solutions in approximately 12 minutes. Further investigations can be conducted into different weight parameters to uncover patterns in solution times.

Table 4: Initial results obtained after running the optimization model with different surrogate models

Model	Design (**u**,**b**)	Runtime (s)	Objective Value	θ (surr, sim)
Linear Regression	[3 3 3 3 3 3 0 0 0 0],[1 1 1 0 1 0 0 0 0 0]	0.0642	14.0478	0.9712, 0.5383
Regression Tree	[3 3 3 3 3 3 3 3 0 0],[0 1 1 0 0 0 1 0 0 0]	1.6391	24.1475	0.95, 0.95
RT (θ) + NN (f_3)	[3 3 3 3 3 3 3 3 0 0],[0 1 1 0 0 1 1 0 0 0]	710.6304	24.3490	0.95, 0.95

Notable results include that optimizations with each model favors unit 3. This result makes sense intuitively, since sag or f_1 has the strongest weight coefficient in the objective function, and sag quickly increases as bending stiffness decreases. Since unit 3 had the highest bending stiffness, it had the lowest sag despite its higher mass. Evaluating these optimization models on individual objective functions (detailed in Appendix E) resulted in different solutions favouring a wider range of units. However, when only sag was optimized, the third unit always was preferred, and when only mass was optimized, the first unit type was always preferred. Similar to the previous optimization experiment, the linear regression surrogate produced infeasible designs, although the decision tree and neural network continued to produce promising results.

The details of the designs produced by the GA and EDA optimizations are included in Appendix F and follow a similar pattern of f and f_1 designs strongly favouring unit 3 and the remaining objectives having more variation in their units selected.

6.2 Optimization Results

Table 5 summarizes the best solutions obtained by each optimization method across all objective functions. All optimization-based approaches substantially outperform the standard modular TDCR design, demonstrating the benefit of algorithmic morphology optimization.

Table 5: Final results for each function and algorithm. (a) objective value (best result for GA and EDA), (b) runtime (units of seconds unless otherwise stated), and (c) reachability (surrogate, simulation).

Function	Metric	GA	EDA	Gurobi	Random	Standard
f	(a)	19.89	**19.88**	24.15	20.68	27.25
	(b)	1.187 ± 0.629 s	1.014±0.579	1.64	300	instant
	(c)	0.96,0.86	0.96,0.95	0.95, 0.95	-,0.95	-,0.95
f_1	(a)	**25.05**	25.32	33.50	27.41	38.17
	(b)	0.347±0.062	0.529±0.108	1.23	300	instant
	(c)	0.96,0.86	1.12,0.95	0.95,0.95	-,0.95	-,0.95
f_2	(a)	**0.492**	**0.492**	**0.492**	**0.49**	0.615
	(b)	0.289±0.029	0.502±0.249	1.04	300	instant
	(c)	0.97,0.88	0.99,0.88	0.95,0.84	-,0.95	-,0.95
f_3	(a)	0.51	0.51	**0.31**	0.40	1.08
	(b)	0.365±0.068	0.517±0.292	1.12	300	instant
	(c)	0.98,0.95	0.98,0.95	0.95,0.95		-, 0.95
f_4	(a)	63.80	64.2	**51.2**	60.4	77.8
	(b)	0.281±0.014	0.441 ± 0.071	0.93	9300	instant
	(c)	0.96,0.95	1.04,0.95	0.95,0.82	-,0.95	-,0.95

Across objectives, GA and EDA exhibit comparable performance considering their objective function values. Both methods outperform Random sampling in all objectives and exceed Gurobi-based optimization in three out of five cases,

f and f_1. For objectives f_3 and f_4, Gurobi achieves the best objective value. GA, EDA, Gurobi and Random are all tied for f_2. Details and a illustration of the optimality gaps can be found in Appendix G.

Reported runtimes for GA, EDA, and Random sampling are not directly comparable to those of solver-based optimization. For stochastic methods, the reported objective values correspond to the best design obtained across multiple independent runs (20 runs for GA and EDA, and 1,810 samples for Random), while runtimes represent per-run execution time. In contrast, Gurobi returns a single solution per run with a deterministic runtime. As such, runtime comparisons should be interpreted in terms of per-evaluation computational effort rather than time-to-guaranteed-optimality.

The runtimes for GA and EDA can be compared and while they achieve very similar objective function values, their convergence behavior differs. EDA converges in fewer generations, stabilizing after approximately two generations, whereas GA typically requires five to six generations to converge (see Appendix F). This faster convergence in generation count suggests that EDA more rapidly exploits promising regions of the design space. However, GA exhibits lower wall-clock runtime for four of the five objectives, as shown in Table 5(b). This discrepancy arises because EDA incurs additional per-generation computational overhead due to the construction and sampling of a probabilistic model at each iteration. Consequently, GA achieves comparable solution quality with lower per-run computational cost, despite requiring more generations to converge.

Table 5(c) highlights discrepancies between surrogate-predicted reachability and simulation-based reachability, particularly for solver-based optimization. In several cases, designs identified as feasible by the surrogate model fail to meet the reachability threshold α when validated in simulation, resulting in false positives. This effect is most pronounced for Gurobi, which strictly enforces the surrogate constraint (predicted reachability of 0.95) but yields only three out of five designs that remain feasible under full simulation.

These discrepancies stem from approximation error and modeling bias in the surrogate reachability model. While the surrogate enables efficient evaluation and preserves relative performance trends between GA and EDA, its predictions should be interpreted as comparative indicators rather than exact feasibility guarantees. In future iterations, this limitation will be addressed through the inclusion of an outer validation loop that confirms design feasibility via simulation prior to acceptance, as well as through improvements to surrogate model fidelity.

7 Conclusion

This work demonstrated the effectiveness of surrogate-assisted optimization for modular tendon-driven continuum robot design. In particular, genetic algorithms outperformed estimation-of-distribution algorithms in solution quality for the weighted multi-objective problem, and structure-aware surrogate models such as CNNs and GNNs provided accurate predictions of reachability and objective values. These results indicate the feasibility of combining learning-based surrogate models with discrete optimization methods for modular TDCR design under reachability constraints.

With additional time, several extensions would strengthen both the learning and optimization components. On the surrogate side, increasing dataset size and incorporating taskspace geometry as an explicit input could enable generalization across workspaces. Exploring alternative architectures such as attention-based models and integrating uncertainty-aware predictors into the optimization loop may improve robustness. On the optimization side, enhancing probabilistic and evolutionary search mechanisms, including multivariate EDA models and adaptive GA operators, could improve global search efficiency in high-dimensional combinatorial design spaces. Investigating predict-then-optimize frameworks with post-hoc regret correction based on surrogate error, as well as warm-start strategies that exploit the sequential and modular structure of TDCR designs, would provide additional methodological improvements. Finally, validating optimized designs in higher fidelity continuum robot simulators that model friction, hysteresis, and distributed loading would enable more realistic performance evaluation and inform improved objective weighting.

References

Matteo Russo, Elie Gautreau, Xavier Bonnet, and Med Amine Laribi. Continuum robots: From conventional to customized performance indicators. *Biomimetics*, 8(2), 2023a. ISSN 2313-7673. doi:10.3390/biomimetics8020147. URL `https://www.mdpi.com/2313-7673/8/2/147`.

Puspita Triana Dewi, Priyanka Rao, and Jessica Burgner-Kahrs. A lightweight modular segment design for tendon-driven continuum robots with pre-programmable stiffness. In *2024 IEEE 7th International Conference on Soft Robotics (RoboSoft)*, pages 531–536, 2024. doi:10.1109/RoboSoft60065.2024.10522016.

H. Cheong, M. Ebrahimi, and T. Duggan. Optimal design of continuum robots with reachability constraints. *IEEE Robotics and Automation Letters*, 6(2):3600–3607, 2021.

Matteo Russo, Elie Gautreau, Xavier Bonnet, and Med Amine Laribi. Continuum robots: From conventional to customized performance indicators. *Biomimetics*, 8(2):147, 2023b.

M. Jabari, C. Visconte, G. Quaglia, and M. A. Laribi. Feasible static workspace optimization of tendon driven continuum robot based on euclidean norm. In *New Trends in Medical and Service Robotics*, volume 186 of *Mechanisms and Machine Science*, pages 489–501. Springer, 2025.

Daniel M. Bodily, Thomas F. Allen, and Marc D. Killpack. Multi-objective design optimization of a soft, pneumatic robot. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1864–1871, 2017. doi:10.1109/ICRA.2017.7989218.

Alexander I. J. Forrester and Andy J. Keane. *Engineering Design via Surrogate Modelling: A Practical Guide*. John Wiley & Sons, 2008.

Tianyi Xie, Hongyuan Li, C Lee Giles, and Zhen Qi. Neural adaptive optimization for combinatorial problems. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Robert J. Webster and Blake A. Jones. Design and kinematic modeling of constant curvature continuum robots: A review. *The International Journal of Robotics Research*, 29(13):1661–1683, 2010.

8 Who Did What

Table 6: Division of Labour among Team Members

Task	**Subtask**	**Primary**	**Secondary**
EDA and GA Algorithm	Implement and refine code	Mubai	Rozilyn
	Integrate surrogate-assisted evaluation loop	Mubai	Hussein
Surrogate Model Development	Dataset generation	Rozilyn	Adam
	GP/NN model training and uncertainty evaluation	Hussein	Rozilyn
Gurobi Implementation	Implement MIP in Gurobi	Adam	Rozilyn
	Incorporate trained surrogate via ML library	Adam	Hussein
Validation and Experiments	Run simulation verification of selected designs	Rozilyn	Mubai
Evaluation and Analysis	Compare convergence, solution quality, and efficiency across methods	Hussein	Mubai
	Analyze surrogate vs. solver trade-offs and performance trends	Mubai	Adam
	Compare optimized and Baseline results	Rozilyn	Hussein

A Modular Curvature Scaling

From Euler–Bernoulli beam theory, the curvature κ of a beam under bending moment M is given by

$$\kappa = \frac{M}{EI},$$

where E is the Young's modulus, I is the second moment of area, and EI is the bending stiffness. Equivalently,

$$M = \kappa EI.$$

For a fixed tendon displacement, the tendon applies approximately the same bending moment to all units within the segment. Curvature is inversely proportional to bending stiffness, a stiffer unit (larger EI) will exhibit a smaller curvature than a more compliant unit under the same actuation. In our project, we sample κ_s with respect to the least stiff unit in segment s. For any other unit t in the same segment, we equate the applied bending moments:

$$M_s = M_t,$$
$$M_s = \kappa_s EI_s,$$
$$M_t = \kappa_t EI_t,$$
$$\kappa_s EI_s = \kappa_t EI_t.$$

Solving for κ_t yields the scaling rule used in our model:

$$\kappa_t = \kappa_s \frac{EI_s}{EI_t}.$$

B Unit Details

Table 7: Flexural rigidity and mass properties of a single modular unit

Unit type	**Weight (g)**	EI **(N m^2)**
Unit 1	6.4	0.0043
Unit 2	8.4	0.0089
Unit 3	9.0	0.0098

C Objective Function Definitions

C.1 Structural Sag Objective

Structural sag is modeled using an Euler–Bernoulli beam approximation under gravity. The sag objective is defined as

$$f_1(D) = \sum_{j=1}^{N} \frac{g\, M_{\text{dist},j} \left(L^{(u_j)}\right)^3}{3\, EI^{(u_j)}},$$

where g is the gravitational acceleration, $L^{(u_j)}$ and $EI^{(u_j)}$ denote the length and bending stiffness of unit u_j, respectively, and $M_{\text{dist},j}$ represents the distributed mass acting on unit j:

$$M_{\text{dist},j} = \sum_{k=j}^{N} w^{(u_k)} + W_{\text{payload}}.$$

C.2 Compactness Objective

The compactness objective penalizes total backbone length and is defined as

$$f_2(D) = \sum_{j=1}^{N} L^{(u_j)}.$$

C.3 Actuation Effort Objective

Actuation effort is evaluated over the set of feasible configurations Q_{feas} that successfully reach workspace voxels. The effort objective is defined as

$$f_3(D, Q_{\text{feas}}) = \frac{1}{N_r} \sum_{r=1}^{N_r} \sum_{j=1}^{N} EI^{(u_j)} L^{(u_j)} \kappa_{r,j}^2,$$

where $\kappa_{r,j}$ denotes the curvature of unit j in the r-th feasible configuration and N_r is the number of reachable voxels.

C.4 Mass Objective

The total mass objective is defined as

$$f_4(D) = \sum_{j=1}^{N} w^{(u_j)},$$

where $w^{(u_j)}$ denotes the mass of unit type u_j.

D Binary Integer Nonlinear Programming Formulation

To enable solver-based optimization over modular TDCR designs, the problem is formulated as a binary integer nonlinear program. Unit selection and segmentation decisions are encoded using binary variables, while reachability and actuation effort are approximated using surrogate predictors.

Let u_{ij} indicate whether unit type j is placed at position i, b_i denote the placement of a segment boundary, and y_i denote whether position i is occupied. The optimization problem is defined as:

$$\begin{aligned}
\min_{u,b,y} \quad & f_1(U,b,y) + w_2 f_2(U,b,y) + w_3 \hat{f}_3(U,b,y) + w_4 f_4(U,b,y) \\
\text{s.t.} \quad & \sum_j u_{ij} = y_i, && \forall i, \\
& b_i \le y_i, && \forall i, \\
& b_i \le y_{i+1}, && \forall i < N, \\
& \sum_i b_i \le B, \\
& \hat{\Theta}(U,b,y) \ge \alpha, \\
& u_{ij}, b_i, y_i \in \{0,1\}.
\end{aligned}$$

E Results Obtained Using Gurobi and Individual Objective Functions

Table 8: Results obtained using individual objectives and the linear surrogate model.

Objective Used	Design (**u**,**b**)	Runtime (s)	θ (surr, sim)	Objective Value
f_1, Sag	[3 3 3 3 3 3 3 0 0 0 0],[0 1 1 1 1 1 0 0 0 0 0]	0.04	1.0181, 0.5322	16.7478
f_2, Length	[1 1 1 1 1 1 1 0 0 0 0],[1 1 1 1 1 0 0 0 0 0 0]	0.02	1.0071, 0.5333	0.369
f_3, Effort	[1 1 1 1 1 1 1 0 0 0 0],[1 1 1 0 1 0 0 0 0 0 0]	0.03	0.9678, 0.5383	0.3969
f_4, Mass	[1 1 1 1 1 1 1 0 0 0 0],[1 1 1 1 0 0 0 0 0 0 0]	0.06	1.0071, 0.5333	38.4

Table 9: Results obtained using individual objectives and the decision tree.

Objective Used	Design (**u**,**b**)	Runtime (s)	θ (surr, sim)	Objective Value
f_1, Sag	[3 3 3 3 3 3 3 3 3 0 0],[0 1 1 1 0 0 0 0 0 0 0]	1.23	0.95, 0.95	33.4956
f_2, Length	[2 3 1 3 1 1 1 1 0 0],[1 0 0 1 1 0 1 0 0 0 0]	1.04	0.95, 0.8406	0.492
f_3, f_1, Effort	[2 3 3 3 3 3 3 1 1 0 0],[0 1 1 0 0 0 0 0 0 0 0]	1.12	0.95, 0.95	0.3095
f_4, Mass	[1 1 1 1 1 1 1 1 1 0 0],[1 0 0 1 0 1 1 0 0 0 0]	0.93	0.95, 0.8221	51.2

Table 10: Results obtained using individual objectives and the neural network.

Objective Used	Design (**u**,**b**)	Runtime (s)	θ (surr, sim)	Objective Value
f_1, Sag	[3 3 3 3 3 3 3 3 0 0],[1 0 0 1 1 0 1 0 0 0]	13.65	0.95, 0.8237	33.4956
f_2, Length	[1 1 1 1 1 3 2 1 0 0],[0 1 0 0 0 0 0 0 0 0]	13.10	0.95, 0.95	0.492
f_3, Effort	-	>2h	-	-
f_4, Mass	[1 1 1 1 1 1 1 1 0 0],[1 0 0 1 1 0 1 0 0 0]	4.93	0.95, 0.8237	51.2

F GA/EDA Results

Method	Obj.	Best Design **u**,**b**	Time (s)	Best Val.	θ (surr,sim)	Obj. Mean ± std.	Theta Mean ± std.
GA	f	**u** =[3 3 3 3 3 3 2 3 3 0 0] **b** =[1 0 1 1 0 0 0 0 0 0]	1.187 ± 0.629 s	19.89	0.955, 0.950	7134.97 ± 26272.48	0.97 ± 0.05
GA	f_1	**u** =[3 3 3 3 3 3 2 3 3 0 0] **b** =[1 0 1 1 0 0 0 0 0 0]	0.292 ± 0.013 s	25.05	0.955, 0.950	7146.07 ± 26273.51	0.98 ± 0.06
GA	f_2	**u** =[3 3 3 3 1 2 3 3 0 0] **b** =[1 0 1 1 0 0 0 0 0 0]	0.289 ± 0.029 s	0.49	0.965, 0.950	2013.31 ± 4996.81	0.99 ± 0.06
GA	f_3	**u** =[2 1 2 3 1 3 1 3 2 1] **b** =[1 0 1 0 1 0 0 0 0 0]	0.365 ± 0.068 s	0.51	0.982, 0.950	461.02 ± 1996.26	0.98 ± 0.03
GA	f_4	**u** =[1 2 3 1 1 3 3 2 0 0] **b** =[0 0 1 1 0 1 0 0 0 0]	0.281 ± 0.014 s	63.800	0.955, 0.950	528.74 ± 1999.03	0.97 ± 0.03
EDA	f	**u** =[2 3 2 3 2 2 3 2 0 0] **b** =[0 1 0 0 1 0 1 0 0 0]	1.207 ± 0.501 s	19.88	0.959, 0.950	15597.78 ± 14895.71	0.96 ± 0.04
EDA	f_1	**u** =[2 3 3 3 3 3 3 2 0 0] **b** =[0 1 0 0 1 0 1 0 0 0]	0.529 ± 0.108 s	25.32	1.117, 0.950	8028.98 ± 21715.94	0.97 ± 0.07
EDA	f_2	**u** =[3 3 3 3 1 2 3 3 0 0] **b** =[1 0 1 1 0 0 0 0 0 0]	0.502 ± 0.249 s	0.49	0.993, 0.950	7991.74 ± 21717.12	0.97 ± 0.07
EDA	f_3	**u** =[2 1 3 1 3 2 3 2 3 0] b =[1 0 1 1 0 0 0 0 0 0]	0.517 ± 0.292 s	0.51	0.982, 0.950	7991.76 ± 21717.12	0.95 ± 0.04
EDA	f_4	**u** =[1 3 1 3 1 3 3 3 0 0] **b** =[1 0 1 1 0 0 0 0 0 0]	0.433 ± 0.088 s	64.200	1.037, 0.950	5699.86 ± 14877.44	0.97 ± 0.05

G Optimality Gap Values

Table 11: Optimality gaps relative to the best feasible solution for each objective. Lower values indicate better performance.

Objective	GA	EDA	Gurobi	Random	Standard
f	0.001	0.000	0.215	0.040	0.371
f_1	0.000	0.011	0.337	0.094	0.524
f_2	0.000	0.000	0.000	0.000	0.250
f_3	0.642	0.642	0.000	0.323	2.484
f_4	0.246	0.254	0.000	0.180	0.520

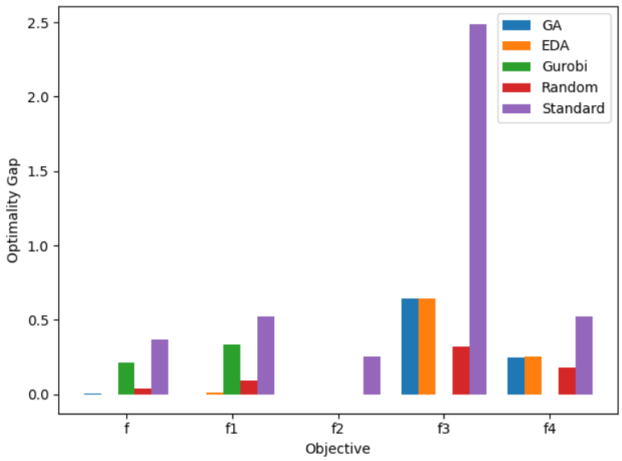


Figure 2: Optimality gaps for each optimization method across objectives, measured relative to the best feasible solution. Lower values indicate solutions closer to the empirical optimum.